



UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

**FACULTAD DE INGENIERÍA DE PRODUCCIÓN
Y SERVICIOS**

**ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS**

Laboratorio de INTRODUCCIÓN AL DESARROLLO WEB

DOCENTE:

CARLO JOSE LUIS CORRALES DELGADO

TEMA:

LABORATORIO 00
GIT Y GITHUB PARA EL DESARROLLO WEB

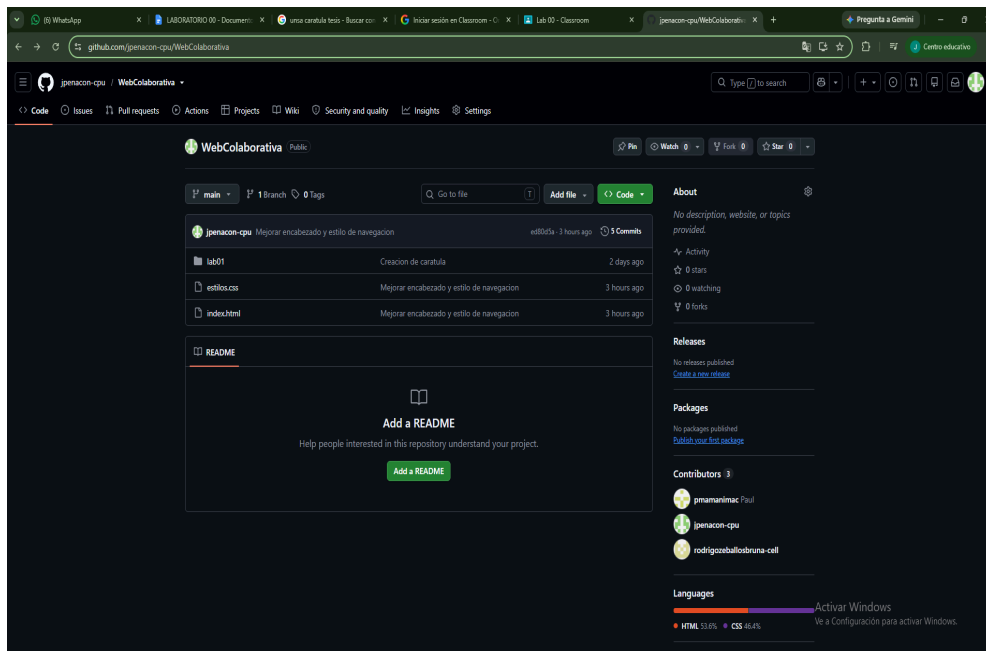
Estudiantes :

- ZEBALLOS BRUNA RODRIGO NELSON
- MAMANI MACHACA PAUL ROEL
 - JAHIR PEÑA CONDORI

AREQUIPA-PERÚ
2026

EVIDENCIAS:

- <https://github.com/jpenacon-cpu/WebColaborativa>
- 3 COLABORADORES:



- HISTORIAL DE COMMITS:

```
C:\Users\Rodrigo Zeballos\Desktop\WebColaborativa>git log
commit c95ad522238da3e323ab015353fc6c6a9fef7d4b (HEAD -> main, origin/main, origin/HEAD)
Author: Rodrigo Zeballos <rodrigo.zeballos.bruna@gmail.com>
Date: Sat Sep 5 21:44:26 2026 -0500

    Informacion Grupo y Pie Página

commit ed80d5a587dd4679f1c95d733a7b874762b6444b
Author: Jahir Peña Condori <jpenacon@unsa.edu.pe>
Date: Sat Sep 5 16:25:28 2026 -0500

    Mejorar encabezado y estilo de navegacion

commit e83fccf9d6e3f4874b2f8dcbe3d04719a93f3f66
Author: Paul Mamani <pmamanimac@unsa.edu.pe>
Date: Sat Sep 5 15:55:21 2026 -0500

    añadiendo título y encabezado

commit 4fb3e2a61bcfb5591846ffcebab883fb32f47bd0
Author: Paul Mamani <pmamanimac@unsa.edu.pe>
Date: Sat Sep 5 00:00:26 2026 -0500

    Agrega descripción del proyecto

commit 547f7f178c6c7a829224b5eb49fdbac5528a29e3
Author: rodrigozeballosbruna-cell <rodrigo.zeballos.bruna@gmail.com>
Date: Thu Sep 3 18:13:47 2026 -0500

    Creacion de caratula
```

- EVIDENCIA DE GIT PULL:

```
C:\IDWeb\WebColaborativa>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   estilos.css
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

C:\IDWeb\WebColaborativa>git add .

C:\IDWeb\WebColaborativa>git commit -m "añadiendo título y encabezado"
[main e83fccf] añadiendo título y encabezado
 2 files changed, 10 insertions(+), 2 deletions(-)

C:\IDWeb\WebColaborativa>git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 627 bytes | 209.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/jpenacon-cpu/WebColaborativa
 4fb3e2a..e83fccf  main -> main
```

```
C:\Windows\system32\cmd.exe: X + v
C:\Users\Usuario\WebColaborativa>git pull origin main
From https://github.com/jpenacon-cpu/WebColaborativa
 * branch      main      -> FETCH_HEAD
Already up to date.

C:\Users\Usuario\WebColaborativa>git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

C:\Users\Usuario\WebColaborativa>git add .

C:\Users\Usuario\WebColaborativa>git commit -m "Mejorar encabezado y estilo de navegacion"
[main ed80d5a] Mejorar encabezado y estilo de navegacion
 2 files changed, 46 insertions(+), 8 deletions(-)

C:\Users\Usuario\WebColaborativa>git push
info: please complete authentication in your browser...
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 868 bytes | 868.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/jpenacon-cpu/WebColaborativa
 e83fccf..ed80d5a  main -> main

C:\Users\Usuario\WebColaborativa>git push origin main
```

```

C:\Users\Rodrigo Zeballos>cd Desktop

C:\Users\Rodrigo Zeballos\Desktop>cd WebColaborativa

C:\Users\Rodrigo Zeballos\Desktop\WebColaborativa>git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 917 bytes | 917.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/jpenacon-cpu/WebColaborativa.git
   ed80d5a..c95ad52  main -> main

C:\Users\Rodrigo Zeballos\Desktop\WebColaborativa>

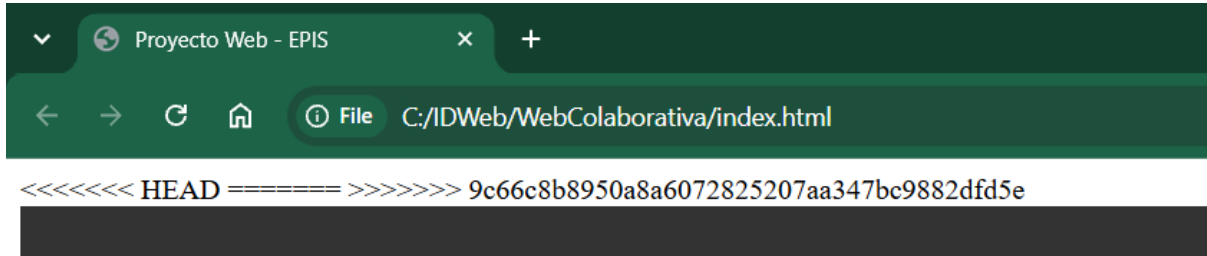
```

- IDENTIFICACIÓN DEL CONFLICTO:

```

C:\IDWeb\WebColaborativa>git push origin main
To https://github.com/jpenacon-cpu/WebColaborativa
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/jpenacon-cpu/WebColaborativa'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

```



-Se borra “<<<,>>,==”

```

5 | <meta name="viewport" content="width=device-width, initial-scale=1.0">
  | Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
6 | <<<<<< HEAD (Current Change)
7 | <title>Proyecto Web - EPIS</title>
8 | =====
9 | <title>Simulación de conflicto</title>
10| >>>>>> 9c66c8b8950a8a6072825207aa347bc9882dfd5e (Incoming Change)
11| <link rel="stylesheet" href="estilos.css">|
12|
13| </head>
14| <body>

```

- RESOLUCIÓN DEL CONFLICTO

```
C:\IDWeb\WebColaborativa>git status
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

C:\IDWeb\WebColaborativa>git add index.html

C:\IDWeb\WebColaborativa>git status
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   index.html
```

```
C:\IDWeb\WebColaborativa>git commit -m "resolver conflict"
[main 1f39db1] resolver conflict

C:\IDWeb\WebColaborativa>git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

C:\IDWeb\WebColaborativa>git push origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 748 bytes | 187.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/jpenacon-cpu/WebColaborativa
  9c66c8b..1f39db1  main -> main
```

PREGUNTAS:

1. ¿Por qué es necesario ejecutar git pull antes de comenzar a trabajar?

Es necesario para traer e integrar los últimos cambios del repositorio remoto a mi rama local. Si no lo hago, estaré trabajando sobre una versión desactualizada del código, lo que aumentará considerablemente la probabilidad de generar conflictos al intentar subir mis cambios más tarde.

2. ¿Qué ocurre cuando dos estudiantes modifican las mismas líneas de un archivo?

Al momento de hacer la integración (git merge o git pull), Git no sabe automáticamente cuál de las dos versiones es la correcta. Por lo tanto, detiene el proceso de combinación y genera un conflicto de fusión (merge conflict) para que los involucrados decidan qué cambios mantener.

3. ¿Qué significa CONFLICT (content)?

Significa que Git detectó modificaciones contradictorias en el contenido interno de uno o varios archivos (por ejemplo, cambios en las mismas líneas) y no pudo resolver la combinación por sí solo. Requiere intervención manual para solucionar la diferencia.

4. ¿Git puede resolver todos los conflictos automáticamente?

Explique.

No. Git es muy inteligente para combinar cambios cuando se realizan en partes o archivos diferentes del proyecto, pero no puede inferir la intención lógica de los programadores cuando dos personas editan el mismo bloque de código. En estos casos, la resolución requiere criterio humano para no romper la funcionalidad.

5. ¿Qué significan los marcadores <<<<<<, ===== y >>>>>>?

Son marcas visuales que Git inserta directamente en el archivo con conflicto para delimitar las diferencias:

<<<<<< HEAD:: Indica el inicio del bloque con los cambios de mi versión actual (mi rama local).

===== : Es el separador entre ambas versiones.

>>>>>> {hash/rama}: Marca el final del bloque con los cambios que vienen de la otra rama o del repositorio remoto.

6. ¿Qué procedimiento debe seguirse para solucionar un conflicto?

1. Identificar los archivos: Revisión.

Ejecutar `git status` para ver qué archivos están marcados como "unmerged".

Editar y elegir código: Resolución.

Abrir los archivos afectados (en VS Code o el editor preferido) y borrar los marcadores (<<<<<<, =====, >>>>>>), dejando únicamente el código final correcto.

Probar el sistema: Verificación.

Asegurarse de que el proyecto compile/ejecute bien con la solución elegida.

Marcar como resuelto: Indexación.

Ejecutar `git add <archivo_resuelto>` para indicarle a Git que el conflicto se solucionó.

Finalizar el proceso: Commit.

Ejecutar `git commit` para registrar el commit de fusión (merge commit).

7. ¿Por qué después de solucionar un conflicto debemos ejecutar git add?

Porque `git add` le notifica a Git que el archivo en cuestión ya fue editado, el conflicto se resolvió manualmente y el estado actual está listo (staging area) para ser empaquetado en el commit de resolución.

8. ¿Qué función cumple el commit que registra la resolución del conflicto?

Cumple la función de consolidar la fusión de ambas ramas, guardando en el historial una "foto" (snapshot) con el estado unificado y coherente del código. Además, sirve como registro histórico de cómo y cuándo se resolvió dicha divergencia.

9. ¿Qué diferencia existe entre trabajar individualmente y trabajar colaborativamente con Git?

Individual: El flujo es lineal. No hay riesgo de que alguien cambie el código remoto mientras tú trabajas, por lo que rara vez te topas con conflictos o la necesidad de sincronizar cambios ajenos.

Grupal: **Colaborativo:** Es un flujo asíncrono y distribuido. Requiere comunicación constante, gestión de ramas independientes (feature branches), sincronización frecuente (git pull), revisiones de código (Pull Requests) y manejo de conflictos de fusión.

10. ¿Qué buenas prácticas recomienda para evitar conflictos innecesarios?

Comunicación constante: Avisar al equipo antes de realizar refactorizaciones grandes o cambios en archivos de configuración compartidos.

Commits atómicos y pequeños: Subir cambios en partes pequeñas y con mensajes claros, en lugar de acumular días de trabajo en un solo commit gigante.

CONCLUSIONES:

1. ¿Cuál es la diferencia conceptual y práctica entre Git y GitHub?

La diferencia más notoria es que uno es la herramienta técnica en la que se puede modificar el código sin necesidad de una conexión a internet y el otro es la plataforma de alojamiento de repositorios creados con Git, funcionando como un entorno colaborativo y visual donde los programadores comparten su código.

2. Explique la diferencia entre un repositorio local y un repositorio remoto.

¿Qué función

cumplen git push y git pull?

El repositorio local es la copia del proyecto guardada en la computadora, el repositorio remoto es la versión del proyecto alojado en un servidor externo.

GIT PUSH: Sube los commits que se registró en el repositorio local hacia el remoto.

GIT PULL: Descargar los últimos cambios del repositorio remoto y los fusiona automáticamente con la rama local.

3. ¿Cuál es la finalidad de git add, git commit y git status? Explique cómo se relacionan

dentro del flujo de trabajo.

Git add: Con este comando podemos añadir las modificaciones realizadas

Git commit: Podemos añadirle una descripción y guardar el último cambio mediante un punto de control al que podríamos volver si fuera necesario

Git status: Muestra información de los archivos modificados

Se relacionan mediante un sistema de pasos a seguir para poder guardar correctamente los cambios que se realicen, siguiendo este orden:

1. Git status
2. Git add
3. Git commit

AUTOEVALUACIÓN :

-ESTUDIANTE A (JAHIR):

Repositorio colaborativo	Excelente
Uso de Git	Excelente
Modificación HTML/CSS	Excelente
Resolución de conflictos	Excelente
Historial de commits	Excelente
Trabajo colaborativo	Excelente

-ESTUDIANTE B (PAUL):

Repositorio colaborativo	Excelente
Uso de Git	Excelente
Modificación HTML/CSS	Excelente
Resolución de conflictos	Excelente
Historial de commits	Excelente
Trabajo colaborativo	Excelente

-ESTUDIANTE C (RODRIGO):

Repositorio colaborativo	Excelente
Uso de Git	Excelente
Modificación HTML/CSS	Excelente
Resolución de conflictos	Excelente
Historial de commits	Excelente
Trabajo colaborativo	Excelente